

Variabili

Prof. Rizzotti

1 Le variabili

Una **variabile** è uno spazio di memoria identificato attraverso un nome, nel quale possiamo memorizzare un valore.

Ad esempio, possiamo utilizzare una variabile per memorizzare l'età di una persona, il risultato di un calcolo oppure un carattere inserito dall'utente.

In C++, ogni variabile possiede un **tipo**, che determina quali valori può contenere e quanta memoria occupa.

2 Dichiarazione e inizializzazione

2.1 Dichiarare una variabile

Per dichiarare una variabile in C++ utilizziamo la seguente sintassi:

```
1 TIPO nome_variabile;
```

Ad esempio:

```
1 int eta;
```

Questa istruzione dichiara una variabile chiamata `eta`, di tipo `int`.

Il tipo `int` indica che la variabile può contenere numeri interi.

2.2 Inizializzare una variabile

L'**inizializzazione** consiste nell'assegnare un valore alla variabile nel momento in cui viene dichiarata.

Ad esempio:

```
1 int eta = 16;
```

In questo caso dichiariamo una variabile di tipo `int` e le assegniamo immediatamente il valore 16.

Possiamo anche dichiarare una variabile e assegnarle un valore successivamente:

```
1 int eta;  
2  
3 eta = 16;
```

La seconda istruzione è un'assegnazione, non un'inizializzazione.

2.3 Attenzione alle variabili non inizializzate

Consideriamo il seguente programma:

```
1 #include <iostream>  
2  
3 int main() {  
4  
5     int numero;  
6  
7     std::cout << numero;  
8  
9     return 0;  
10 }
```

Abbiamo dichiarato la variabile `numero`, ma non le abbiamo assegnato alcun valore.

Una variabile locale di questo tipo non viene automaticamente inizializzata a zero.

Possiamo immaginare che lo spazio di memoria riservato alla variabile contenga già una sequenza di cifre binarie, lasciata da precedenti operazioni.

Se non inizializziamo la variabile, non abbiamo stabilito quale valore debba contenere.

Nel nostro esempio, leggere una variabile `int` locale non inizializzata produce un comportamento non definito: il programma potrebbe apparentemente mostrare un numero casuale, ma non possiamo prevedere il risultato.

Per questo motivo, è fondamentale inizializzare sempre le variabili prima di utilizzarle.

Ad esempio:

```
1 int numero = 0;
```

Questa semplice abitudine permette di evitare numerosi bug e malfunzionamenti.

3 I principali tipi di dato

In C++ esistono diversi tipi di dato.

Tra quelli che utilizzeremo troviamo:

- `int`
- `long`
- `float`
- `double`
- `char`
- `bool`
- `void`

Ogni tipo ha caratteristiche differenti e può occupare una diversa quantità di memoria.

Ricordiamo che:

$$1 \text{ byte} = 8 \text{ bit}$$

Le dimensioni di alcuni tipi possono variare a seconda del compilatore e del sistema utilizzato.

Per semplicità, negli esempi considereremo un sistema nel quale un byte contiene 8 bit e un `int` occupa 32 bit.

4 Il tipo `int`

Il tipo `int` viene utilizzato per rappresentare numeri interi, positivi e negativi.

Ad esempio:

```
1 int eta = 16;
2
3 int temperatura = -5;
4
5 int numero = 100;
```

Su molti sistemi moderni, un `int` occupa:

$$32 \text{ bit} = 4 \text{ byte}$$

4.1 Quanti numeri possiamo rappresentare?

Con 32 bit possiamo costruire:

$$2^{32} = 4\,294\,967\,296$$

combinazioni binarie differenti.

Se utilizzassimo tutti i 32 bit per rappresentare esclusivamente numeri interi non negativi, il valore massimo sarebbe:

$$2^{32} - 1 = 4\,294\,967\,295$$

Infatti, il numero binario più grande sarebbe formato da 32 bit tutti uguali a 1:

$$11111111111111111111111111111111$$

Tuttavia, il tipo `int` deve rappresentare anche numeri negativi.

4.2 Il bit di segno

Nella rappresentazione degli interi con segno, il **bit più significativo** è il bit più a sinistra.

Nella rappresentazione in complemento a due, utilizzata dai moderni sistemi C++, questo bit indica anche il segno del numero:

- se vale 0, il numero è non negativo;
- se vale 1, il numero è negativo.

Per rappresentare il numero positivo più grande, dobbiamo quindi impostare il bit più significativo a 0 e tutti gli altri 31 bit a 1.

Otteniamo:

$$01111111111111111111111111111111$$

Questo numero corrisponde a:

$$2^{31} - 1 = 2\,147\,483\,647$$

Possiamo capire il motivo osservando che il numero:

$$10000000000000000000000000000000$$

se interpretato come numero senza segno, rappresenta:

$$2^{31}$$

Il numero positivo più grande rappresentabile con un `int` a 32 bit è quindi:

$$\boxed{2^{31} - 1}$$

Il valore minimo è invece:

$$\boxed{-2^{31}}$$

Pertanto, l'intervallo è:

$$-2\,147\,483\,648 \leq x \leq 2\,147\,483\,647$$

Il numero minimo ha un valore assoluto maggiore di uno rispetto al massimo, perché lo zero occupa una delle combinazioni non negative.

5 Il tipo unsigned

La parola chiave `unsigned` permette di dichiarare tipi interi che rappresentano esclusivamente valori non negativi.

Ad esempio:

```
1 unsigned int numero = 100;
```

Possiamo anche scrivere:

```
1 unsigned numero = 100;
```

In questo caso, non dobbiamo più riservare il bit più significativo per distinguere i numeri positivi da quelli negativi.

Tutti i 32 bit possono contribuire alla rappresentazione del valore.

L'intervallo di un `unsigned int` a 32 bit è:

$$0 \leq x \leq 2^{32} - 1$$

ovvero:

$$0 \leq x \leq 4\,294\,967\,295$$

Il valore massimo è rappresentato da 32 bit tutti uguali a 1.

11111111111111111111111111111111

Attenzione: `unsigned` non aumenta la quantità di memoria occupata dalla variabile.

Permette semplicemente di utilizzare tutte le combinazioni disponibili per rappresentare numeri non negativi.

Può essere utilizzato anche con altri tipi interi, come `long`.

6 Il tipo long

Il tipo `long` viene utilizzato per rappresentare numeri interi.

La sua dimensione dipende dal sistema e dal compilatore.

Su alcuni sistemi occupa:

$$32 \text{ bit} = 4 \text{ byte}$$

Su altri sistemi occupa:

$$64 \text{ bit} = 8 \text{ byte}$$

Ad esempio:

```
1 long popolazione = 1000000;
```

Un `long` a 64 bit può rappresentare valori compresi tra:

$$-2^{63}$$

e:

$$2^{63} - 1$$

Un `long` a 32 bit, invece, ha lo stesso intervallo di un `int` a 32 bit.

Se abbiamo bisogno di un tipo intero che occupi almeno 64 bit, possiamo utilizzare:

```
1 long long numero = 10000000000LL;
```

Il tipo `long long` occupa almeno 64 bit.

7 Precisione degli interi

Quando parliamo di numeri interi, una maggiore quantità di bit permette di rappresentare un intervallo più ampio di valori.

Ad esempio, confrontiamo due tipi interi con segno:

- un intero a 32 bit;
- un intero a 64 bit.

Il primo può rappresentare valori compresi tra:

$$-2^{31} \text{ e } 2^{31} - 1$$

Il secondo può rappresentare valori compresi tra:

$$-2^{63} \text{ e } 2^{63} - 1$$

Con 64 bit possiamo quindi rappresentare numeri molto più grandi.

Gli interi rappresentabili sono comunque esatti: non esiste un'approssimazione delle cifre come accade con i numeri in virgola mobile.

8 I tipi float e double

Finora abbiamo considerato solamente numeri interi.

Se vogliamo rappresentare numeri con una parte decimale, possiamo utilizzare i tipi:

- float
- double

Ad esempio:

```
1 float temperatura = 21.5f;  
2  
3 double pi_greco = 3.141592653589793;
```

Questi tipi utilizzano una rappresentazione chiamata **virgola mobile** (floating point). Non analizzeremo nel dettaglio come vengono memorizzati internamente questi numeri. Ci interessa soprattutto comprendere la differenza tra singola e doppia precisione.

8.1 Singola precisione: float

Il tipo float occupa generalmente:

$$32 \text{ bit} = 4 \text{ byte}$$

Viene chiamato tipo a **singola precisione**.

Nella comune rappresentazione IEEE 754, permette di memorizzare circa 6–7 cifre decimali significative.

Ad esempio:

```
1 float numero = 3.141592f;
```

Il numero viene memorizzato con una precisione limitata.

Non tutti i numeri reali possono essere rappresentati esattamente.

8.2 Doppia precisione: double

Il tipo `double` occupa generalmente:

$$64 \text{ bit} = 8 \text{ byte}$$

Viene chiamato tipo a **doppia precisione**.

Nella comune rappresentazione IEEE 754, permette di memorizzare circa 15–16 cifre decimali significative.

Ad esempio:

```
1 double numero = 3.141592653589793;
```

Rispetto a `float`, il tipo `double` permette di rappresentare un numero maggiore di cifre significative.

8.3 Differenza tra singola e doppia precisione

Possiamo immaginare i numeri reali disposti su una retta.

I numeri rappresentabili da un computer corrispondono solamente ad alcuni punti di questa retta.

Utilizzando più bit possiamo rappresentare un maggior numero di valori.

In questo senso, una maggiore precisione permette di distinguere punti più vicini tra loro.

Il tipo `double`, avendo generalmente 64 bit, offre una precisione maggiore rispetto al tipo `float`, che ne possiede 32.

Attenzione: questo non significa che possiamo memorizzare sempre un numero fisso di cifre dopo la virgola.

La precisione riguarda il numero complessivo di cifre significative e dipende anche dalla grandezza del numero rappresentato.

Ad esempio, se un numero è molto grande, la distanza tra due valori rappresentabili consecutivi può essere maggiore.

Inoltre, alcuni numeri decimali, come 0.1, non possono essere rappresentati esattamente in virgola mobile binaria.

Per questo motivo, i calcoli con `float` e `double` possono produrre piccoli errori di approssimazione.

9 Il tipo char

Il tipo `char` viene utilizzato per rappresentare caratteri.

Ad esempio:

```
1 char lettera = 'A';
2
3 char simbolo = '!';
4
5 char cifra = '7';
```

Un `char` occupa sempre un byte.

Nei sistemi che stiamo considerando:

$$1 \text{ byte} = 8 \text{ bit}$$

Un carattere viene memorizzato attraverso un valore numerico associato a una codifica.

Ad esempio, nella codifica ASCII, la lettera maiuscola A corrisponde al numero 65.

Attenzione: i caratteri vengono racchiusi tra apici singoli.

```
1 char lettera = 'A';
```

Non bisogna confondere un singolo carattere con una stringa di testo.

10 Il tipo bool

Il tipo booleano in C++ si chiama `bool`.

Una variabile di questo tipo può assumere solamente due valori:

- `true`, cioè vero;
- `false`, cioè falso.

Ad esempio:

```
1 bool acceso = true;
2
3 bool maggiorenne = false;
```

Il tipo `bool` occupa generalmente:

8 bit = 1 byte

Dal punto di vista logico, sarebbe sufficiente un solo bit per distinguere due valori.

Tuttavia, nei sistemi comuni, una variabile `bool` occupa un byte.

Il tipo booleano è particolarmente utile nelle condizioni:

```
1 bool acceso = true;
2
3 if (acceso) {
4     std::cout << "Il dispositivo e' acceso";
5 }
```

11 Il tipo void

Il tipo `void` rappresenta l'assenza di un valore.

Viene utilizzato soprattutto quando una funzione non deve restituire nulla.

Ad esempio:

```
1 void saluta() {
2
3     std::cout << "Ciao!";
4
5 }
```

La funzione `saluta` esegue un'operazione, ma non restituisce alcun valore.

A differenza degli altri tipi analizzati, `void` non può essere utilizzato per dichiarare una normale variabile.

Non ha quindi una dimensione in bit o byte associata a un valore memorizzabile.

12 Il tipo di ritorno della funzione main

Nei nostri programmi abbiamo sempre utilizzato la funzione:

```
1 int main() {
2
3     return 0;
4 }
```

Adesso possiamo comprendere il significato della parola `int`.

Il tipo scritto prima del nome di una funzione indica il suo **tipo di ritorno**.

Nel caso della funzione `main`, il tipo di ritorno è `int`.

Questo significa che la funzione restituisce un numero intero.

L'istruzione:

```
1 return 0;
```

termina la funzione e restituisce il valore 0.

Nel caso della funzione `main`, questo valore viene comunicato al sistema operativo.

Per convenzione, restituire 0 significa che il programma è terminato correttamente.

Possiamo quindi interpretare:

```
1 return 0;
```

come un messaggio al sistema operativo:

Il programma è terminato senza segnalare errori.

13 Overflow

Ogni tipo numerico può rappresentare solamente un determinato intervallo di valori.

Cosa succede se proviamo a rappresentare un numero che supera questo intervallo?

Questo fenomeno prende il nome di **overflow**.

13.1 Overflow degli interi

Consideriamo un intero con segno a 32 bit.

Il suo valore massimo è:

$$2^{31} - 1 = 2\,147\,483\,647$$

Supponiamo di scrivere:

```
1 int numero = 2147483647;
2
3 numero = numero + 1;
```

Il risultato matematico sarebbe:

$$2\,147\,483\,648$$

Tuttavia, questo numero non è rappresentabile con un `int` a 32 bit.

Si verifica quindi un overflow.

Nel caso degli interi con segno, l'overflow produce un comportamento non definito.

Non possiamo quindi fare affidamento sul risultato del programma.

Su alcuni sistemi potremmo osservare un valore negativo, ma questo comportamento non è garantito.

13.2 Overflow degli interi unsigned

Nel caso degli interi `unsigned`, il comportamento è differente.

Consideriamo un `unsigned int` a 32 bit:

```
1 unsigned int numero = 4294967295U;
2
3 numero = numero + 1;
```

Il valore iniziale è il massimo rappresentabile:

$$2^{32} - 1$$

Aggiungendo 1, il risultato torna a 0.

Infatti, gli interi `unsigned` seguono un'aritmetica modulare.

Nel nostro esempio:

$$(2^{32} - 1 + 1) \bmod 2^{32} = 0$$

Questo comportamento è definito dal linguaggio C++.

13.3 Overflow dei numeri in virgola mobile

Anche i tipi `float` e `double` hanno un limite ai valori rappresentabili.

Se il risultato di un'operazione supera questo limite, si può verificare un overflow.

Nei comuni sistemi che utilizzano IEEE 754, il risultato può diventare un valore speciale chiamato infinito.

Non bisogna confondere l'overflow con i piccoli errori di approssimazione dovuti alla precisione limitata dei numeri in virgola mobile.

14 Dimensioni dei principali tipi

La seguente tabella riporta le dimensioni comunemente utilizzate nei sistemi moderni.

Tipo	Bit	Byte
<code>int</code>	32	4
<code>long</code>	32 o 64	4 o 8
<code>long long</code>	64	8
<code>float</code>	32	4
<code>double</code>	64	8
<code>char</code>	8	1
<code>bool</code>	8	1
<code>void</code>	—	—

Le dimensioni effettive dipendono dall'implementazione del linguaggio.

In particolare, `long` può occupare 32 oppure 64 bit, mentre `char` occupa sempre un byte, che non è necessariamente composto da 8 bit su ogni sistema.

¹

¹Parti di questo documento sono output di LLM.